JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.stream

Class StreamSupport

java.lang.Object
    java.util.stream.StreamSupport

```
public final class StreamSupport
extends Object
```

Low-level utility methods for creating and manipulating streams.

This class is mostly for library writers presenting stream views of data structures; most static stream methods intended for end users are in the various `Stream` classes.

Since:

1.8

Method Summary

Concrete Methods

Modifier and Type	Method	Description
static DoubleStream	doubleStream(Spliterator.OfDouble spliterator, boolean parallel)	Creates a new sequential or DoubleStream Spliterator
static DoubleStream	doubleStream(Supplier<? extends Spliterator.OfDouble> supplier, int characteristics, boolean parallel)	Creates a new sequential or DoubleStream Supplier of Spliterator

static IntStream	intStream(Spliterator.OfInt spliterator, boolean parallel)	Creates a new sequential or IntStream fr Spliterator
static IntStream	intStream(Supplier<? extends Spliterator.OfInt> supplier, int characteristics, boolean parallel)	Creates a new sequential or IntStream fr Supplier of Spliterator
static LongStream	longStream(Spliterator.OfLong spliterator, boolean parallel)	Creates a new sequential or LongStream f Spliterator
static LongStream	longStream(Supplier<? extends Spliterator.OfLong> supplier, int characteristics, boolean parallel)	Creates a new sequential or LongStream f Supplier of Spliterator
static <T> Stream<T>	stream(Spliterator<T> spliterator, boolean parallel)	Creates a new sequential or Stream from Spliterator
static <T> Stream<T>	stream(Supplier<? extends Spliterator<T>> supplier, int characteristics, boolean parallel)	Creates a new sequential or Stream from Supplier of Spliterator

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail

stream

```
public static <T> Stream<T> stream(Spliterator<T> spliterator,
                                   boolean parallel)
```

Creates a new sequential or parallel Stream from a Spliterator.

The spliterator is only traversed, split, or queried for estimated size after the terminal operation of the stream pipeline commences.

It is strongly recommended the spliterator report a characteristic of `IMMUTABLE` or `CONCURRENT`, or be late-binding. Otherwise, `stream(java.util.function.Supplier, int, boolean)` should be used to reduce the scope of potential interference with the source. See Non-Interference for more details.

Type Parameters:

`T - the type of stream elements`

Parameters:

`spliterator - a Spliterator describing the stream elements`

`parallel - if true then the returned stream is a parallel stream; if false the returned stream is a sequential stream.`

Returns:

`a new sequential or parallel Stream`

stream

```
public static <T> Stream<T> stream(Supplier<? extends Spliterator<T>> supplier,
                                   int characteristics,
                                   boolean parallel)
```

Creates a new sequential or parallel `Stream` from a `Supplier` of `Spliterator`.

The `Supplier.get()` method will be invoked on the supplier no more than once, and only after the terminal operation of the stream pipeline commences.

For spliterators that report a characteristic of `IMMUTABLE` or `CONCURRENT`, or that are late-binding, it is likely more efficient to use `stream(java.util.Spliterator, boolean)` instead.

The use of a `Supplier` in this form provides a level of indirection that reduces the scope of potential interference with the source. Since the supplier is only invoked after the terminal operation commences, any modifications to the source up to the start of the terminal operation are reflected in the stream result. See Non-Interference for more details.

Type Parameters:

`T - the type of stream elements`

Parameters:

`supplier - a Supplier of a Spliterator`

`characteristics - Spliterator characteristics of the supplied Spliterator. The characteristics must be equal to`

supplier.get().characteristics(), otherwise undefined behavior may occur when terminal operation commences.

parallel - if true then the returned stream is a parallel stream; if false the returned stream is a sequential stream.

Returns:

a new sequential or parallel Stream

See Also:

stream(java.util.Spliterator, boolean)

intStream

```
public static IntStream intStream(Spliterator.OfInt spliterator,
                                  boolean parallel)
```

Creates a new sequential or parallel IntStream from a Spliterator.OfInt.

The spliterator is only traversed, split, or queried for estimated size after the terminal operation of the stream pipeline commences.

It is strongly recommended the spliterator report a characteristic of IMMUTABLE or CONCURRENT, or be late-binding. Otherwise, intStream(java.util.function.Supplier, int, boolean) should be used to reduce the scope of potential interference with the source. See Non-Interference for more details.

Parameters:

spliterator - a Spliterator.OfInt describing the stream elements

parallel - if true then the returned stream is a parallel stream; if false the returned stream is a sequential stream.

Returns:

a new sequential or parallel IntStream

intStream

```
public static IntStream intStream(Supplier<? extends Spliterator.OfInt> supplier,
                                  int characteristics,
                                  boolean parallel)
```

Creates a new sequential or parallel IntStream from a Supplier of Spliterator.OfInt.

The Supplier.get() method will be invoked on the supplier no more than once, and only after the terminal operation of the stream pipeline commences.

For spliterators that report a characteristic of IMMUTABLE or CONCURRENT, or that are late-binding, it is likely more efficient to use intStream(java.util.Spliterator.OfInt, boolean) instead.

The use of a Supplier in this form provides a level of indirection that reduces the scope of potential interference with the source. Since the supplier is only invoked after the terminal operation commences, any modifications to the source up to the start of the terminal operation are reflected in the stream result. See Non-Interference for more details.

Parameters:

supplier - a Supplier of a Spliterator.OfInt

characteristics - Spliterator characteristics of the supplied Spliterator.OfInt. The characteristics must be equal to supplier.get().characteristics(), otherwise undefined behavior may occur when terminal operation commences.

parallel - if true then the returned stream is a parallel stream; if false the returned stream is a sequential stream.

Returns:

a new sequential or parallel IntStream

See Also:

intStream(java.util.Spliterator.OfInt, boolean)

longStream

```
public static LongStream longStream(Spliterator.OfLong spliterator,
                                    boolean parallel)
```

Creates a new sequential or parallel `LongStream` from a `Spliterator.OfLong`.

The spliterator is only traversed, split, or queried for estimated size after the terminal operation of the stream pipeline commences.

It is strongly recommended the spliterator report a characteristic of `IMMUTABLE` or `CONCURRENT`, or be late-binding. Otherwise, `longStream(java.util.function.Supplier, int, boolean)` should be used to reduce the scope of potential interference with the source. See Non-Interference for more details.

Parameters:

`spliterator - a Spliterator.OfLong describing the stream elements`

`parallel - if true then the returned stream is a parallel stream; if false the returned stream is a sequential stream.`

Returns:

`a new sequential or parallel LongStream`

longStream

```
public static LongStream longStream(Supplier<? extends Spliterator.OfLong> suppli
                                    int characteristics,
                                    boolean parallel)
```

Creates a new sequential or parallel `LongStream` from a `Supplier` of `Spliterator.OfLong`.

The `Supplier.get()` method will be invoked on the supplier no more than once, and only after the terminal operation of the stream pipeline commences.

For spliterators that report a characteristic of `IMMUTABLE` or `CONCURRENT`, or that are late-binding, it is likely more efficient to use `longStream(java.util.Spliterator.OfLong, boolean)` instead.

The use of a `Supplier` in this form provides a level of indirection that reduces the scope of potential interference with the source. Since the supplier is only invoked after the terminal operation commences, any modifications to the source up to the start of the terminal operation are reflected in the stream result. See Non-Interference for more details.

Parameters:

supplier - a Supplier of a Spliterator.OfLong

characteristics - Spliterator characteristics of the supplied Spliterator.OfLong. The characteristics must be equal to supplier.get().characteristics(), otherwise undefined behavior may occur when terminal operation commences.

parallel - if true then the returned stream is a parallel stream; if false the returned stream is a sequential stream.

Returns:

a new sequential or parallel LongStream

See Also:

longStream(java.util.Spliterator.OfLong, boolean)

doubleStream

```
public static DoubleStream doubleStream(Spliterator.OfDouble spliterator,
                                        boolean parallel)
```

Creates a new sequential or parallel DoubleStream from a Spliterator.OfDouble.

The spliterator is only traversed, split, or queried for estimated size after the terminal operation of the stream pipeline commences.

It is strongly recommended the spliterator report a characteristic of IMMUTABLE or CONCURRENT, or be late-binding. Otherwise, doubleStream(java.util.function.Supplier, int, boolean) should be used to reduce the scope of potential interference with the source. See Non-Interference for more details.

Parameters:

spliterator - A Spliterator.OfDouble describing the stream elements

parallel - if true then the returned stream is a parallel stream; if false the returned stream is a sequential stream.

Returns:

a new sequential or parallel DoubleStream

doubleStream

```
public static DoubleStream doubleStream(Supplier<? extends Spliterator.OfDouble>
                                        int characteristics,
                                        boolean parallel)
```

Creates a new sequential or parallel `DoubleStream` from a `Supplier` of `Spliterator.OfDouble`.

The `Supplier.get()` method will be invoked on the supplier no more than once, and only after the terminal operation of the stream pipeline commences.

For spliterators that report a characteristic of `IMMUTABLE` or `CONCURRENT`, or that are late-binding, it is likely more efficient to use `doubleStream(java.util.Spliterator.OfDouble, boolean)` instead.

The use of a `Supplier` in this form provides a level of indirection that reduces the scope of potential interference with the source. Since the supplier is only invoked after the terminal operation commences, any modifications to the source up to the start of the terminal operation are reflected in the stream result. See Non-Interference for more details.

Parameters:

`supplier - A Supplier of a Spliterator.OfDouble`

`characteristics - Spliterator characteristics of the supplied Spliterator.OfDouble. The characteristics must be equal to supplier.get().characteristics(), otherwise undefined behavior may occur when terminal operation commences.`

`parallel - if true then the returned stream is a parallel stream; if false the returned stream is a sequential stream.`

Returns:

`a new sequential or parallel DoubleStream`

See Also:

`doubleStream(java.util.Spliterator.OfDouble, boolean)`

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.